3D Roguelike RPG Complete System: TPS, Enemy AI, Puzzles, Camera and Props

- Documentation -

ABOUT THIS DOCUMENT

This is the main documentation of the package. It explains every system that the package offers: the hero, the cameras, combat, the enemies, dialogues, puzzles, props and traps. The inventory, the save and load system, the main menu and the minimap are explained in the second document, **Documentation_V2_Update.pdf**, so please keep both of them together.

Everything described here is already set up in the demo scene, so you can open it and press Play to try it straight away. All screenshots in this document were taken from the current version of the package.

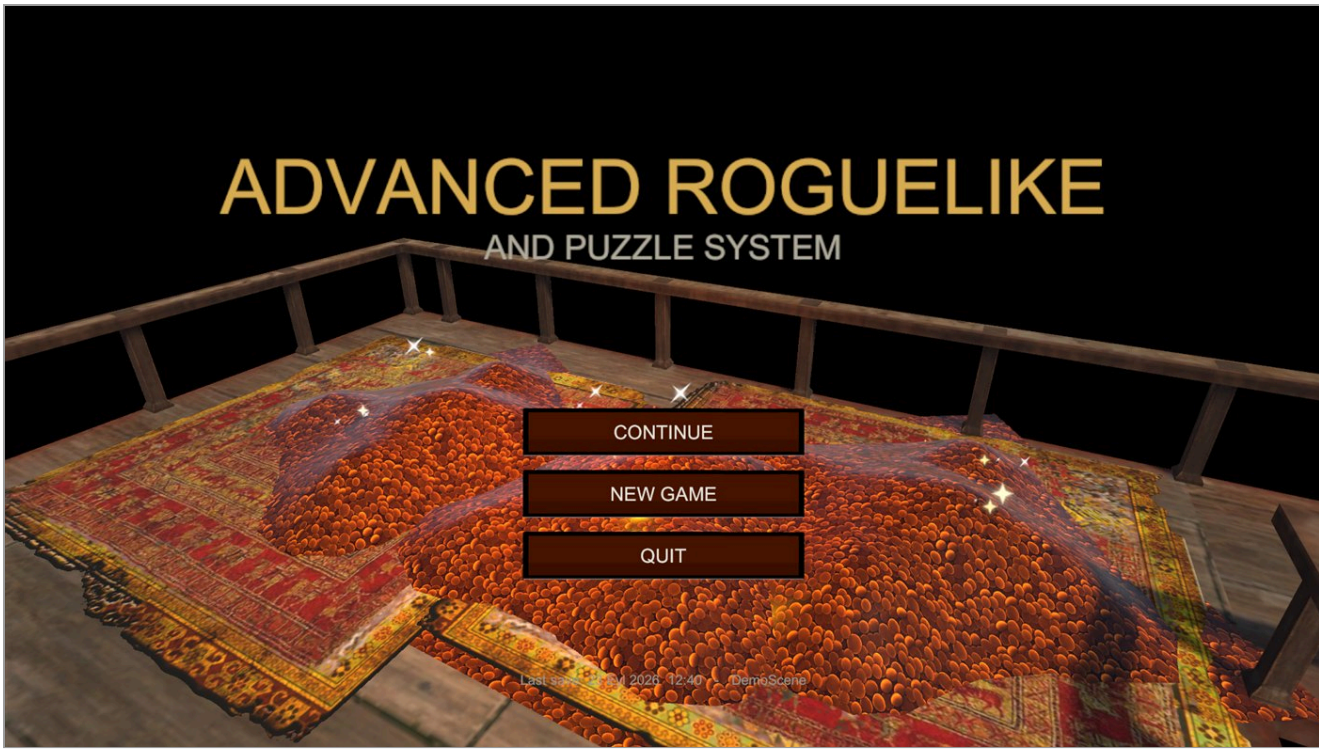
Requirement	Value
Unity version	Unity 6 (6000.3.2f1)
Render pipeline	Built-in Render Pipeline
Input	Input System package. The old Input Manager is switched off in Project Settings.
Platforms	PC (keyboard, mouse and gamepad) and Mobile (on-screen joystick and buttons)

If you have further questions, you can contact with me via queendeveloper95@gmail.com.

INSTALLATION

You can purchase the package from Unity Asset Store and download it by using Package Manager in Unity. After downloading the package, import it into your project and the **AdvancedRogueLikeandPuzzleSystem** folder will appear in your project. Everything is prepared as a prefab in the Prefabs folder, and the scenes are ready in the Scene folder:

Scene	What it shows
MenuScene	The main menu with Continue, New Game and Quit. It is the first scene in the Build Settings.
DemoScene	The full demo level. Every system in this document can be tried here.
DemoScene_Spawner	A small test level where the Enemy Spawner creates waves of enemies.



MenuScene

> **Please note:** the package uses the new Input System only. If you import it into an existing project, open **Edit > Project Settings > Player > Active Input Handling** and choose **Input System Package (New)** or **Both**.

PREFABS TO USE

The package has got lots of prefabs ready to use. You can find all of them archived carefully in the `Assets\AdvancedRogueLikeandPuzzleSystem\Prefabs` directory. You only need to drag and drop them from the directory into your scene, or you can copy them from the demo scene.

Folder	Contents
Character	AdvancedTPSPlayer (your hero), Main Camera and the Archer character model.
Doors	Door and ThroneDoor.
Enemy	Enemy_Archer and three levels of Enemy_Soldier.
Environment	GroundArea, WaterToSwim and two bridges.
Managers	GameManager, EnemySpawner, DialogueTrigger and GameCanvas.
Particles	Blood, impact, fire, explosion, sparkle and sword slash effects.
Props	Potions, key, chest, barrel, lift, torches, stairs, columns and furniture.
Puzzles	Button, Mirror, Gear For Mirror and the ready Puzzle_Mirror setup.
Traps	Six ready traps.
UI	Joystick and Touchpad for mobile controls.
Weapons	Sword, Axe, Shield and the Arrow used by archers.

GAME MANAGER

The Game Manager prefab is the most important prefab in the game. Your hero, your cameras, your Game Canvas and all input settings are inside this one prefab. Drag and drop it into your scene and it will start to work immediately. When a required object is missing, the Console tells you exactly what to add, with messages that start with `[ARPS]`.

```
GameManager
    GameCanvas
        GamePlayPanel, PausePanel, GameOverPanel,
        MinimapPanel, InventoryPanel, FullMapPanel
    CameraFPS
    AdvancedTPSPlayer
        PlayerCameraRoot, Hero, WaterSplash,
        HittingParticle1, HittingParticle2, HittingParticle3
    CameraTPS
    EventSystem
```

- **GameCanvas** contains all UI elements: HUD, pause menu, game over panel, inventory and minimap.
- **AdvancedTPSPlayer** is your hero with its mesh, colliders and components.
- **CameraTPS** and **CameraFPS** are the third person and first person cameras.

The Game Manager object itself also carries the Audio Manager, the Inventory System and the Game Session (save system) components. On the **Game Manager** component you can set the controller type, the starting camera and your keyboard keys:

Field	Default	Meaning
Controller Type	Keyboard Mouse	Keyboard Mouse for PC and gamepad, Mobile for touch screens.
Camera Type	TPS	The camera the game starts with. The player can switch at any time.
Keycode_Interact	E	Pick up, open doors and chests, use levers and the mirror gear.
Keycode_Sprint	Left Shift	Sprint and dash.
Keycode_Jump	Space	Jump.
Keycode_Camera Switch	C	Switch between TPS and FPS cameras.
Keycode_Using_Potion_Heal	Alpha 1	Drink a Health potion.
Keycode_Using_Potion_Mana	Alpha 2	Drink a Mana potion.
Keycode_Inventory	Tab	Open and close the inventory.
Keycode_Map	M	Open and close the full screen map.

Below the keys you will see the Input Actions (Move Action, Jump Action and so on). You do not need to fill them. When an action is left empty, the Game Manager builds it for you from the key above, and adds the gamepad and touch bindings too. If you want your own bindings, just add them to the action in the inspector and the Game Manager will use yours instead.

CONTROLS

Action	Keyboard / Mouse	Gamepad	Mobile
Move	W A S D	Left Stick	Joystick
Look	Mouse	Right Stick	Swipe on the screen
Attack	Left Mouse Button	Right Trigger	Sword button
Shield	Hold Right Mouse Button	Hold Left Trigger	Shield button
Interact	E	South button (A / Cross)	Interact button
Sprint / Dash	Left Shift	Left Stick press	Sprint button
Jump	Space	North button (Y / Triangle)	Jump button
Switch Camera	C	East button (B / Circle)	-
Health / Mana Potion	1 / 2	D-pad Left / Right	Bottle icons
Inventory	Tab	Select	Bag button
Full Screen Map	M	Right Stick press	-
Camera Zoom	Mouse Wheel	D-pad Up / Down	-
Pause	Esc	Start	Pause button

HUD AND MENUS

The HUD shows your hero's portrait and name, the red Health bar and the blue Mana bar, the two potion bottles with their key numbers, your coins, and the minimap in the top right corner. When you get hit, the screen flashes red and the camera shakes.



The HUD in the demo scene

Press **Esc** to pause the game. The pause menu has a Save button (the disk icon), **Continue**, **Restart** which reloads the level, and **Quit** which returns to the main menu. When your health reaches zero, the Game Over panel appears after three seconds with Restart and Quit.



Pause menu



Game Over panel

YOUR HERO: THIRD PERSON CONTROLLER

The package has got a complex and advanced TPS controller. Your hero is the **AdvancedTPSPlayer** object and it uses two main components: **Hero Controller** for everything about gameplay, and **Third Person Controller** for movement. You can set your character's main properties on the Hero Controller component:

Field	Default	Meaning
Name, Profile Image	HERO	Shown on the HUD.
Health	100	Starting and maximum health.
Mana	100	Starting and maximum mana.
Mana Regen Per Tick	5	Mana regenerated every 1.5 seconds.
Speed	6	Walking speed.
Sprint Speed	10	Running speed while Sprint is held.
Hitting Period	0.3	How long the hero stands still after a swing. The third hit of the combo waits twice as long.
Max Health Pot, Max Mana Pot	4	How many bottles the HUD icons represent.
Speed Potion Duration	15	Seconds the Speed potion lasts.
Coin, Level	0, 1	Your coin and level values. Coins are shown on the HUD and can be given as rewards with `HeroController.instance.UpdateCoin(amount)`.

> **Please note:** the Speed and Sprint Speed values on the Hero Controller are copied into the Third Person Controller when the game starts, so change them on the Hero Controller. The damage your hero deals comes from the weapon you are holding (see Combat and Weapons), not from the Hit Power field.

On the **Third Person Controller** you can tune the jump (Jump Height 3, Gravity -15), the ground check and the dash. Every time you press Sprint while moving, the hero dashes forward with a trail effect and spends 20 mana (**Mana Spending_Sprint**). The hero can also swim: any trigger with the `WaterForSwimming` tag, like the WaterToSwim prefab, turns on the swimming animation and the water splash effect.

CAMERAS

There are two cameras and the player can switch between them with **C**. The Third Person camera is the default. In First Person mode the hero's head is hidden so it does not block the view, and holding the shield zooms the camera in.



First person camera, looking at the weapon boards

The **Advanced TPS Camera** component on **CameraTPS** follows the **PlayerCameraRoot** of your hero. The most useful fields are:

Field	Default	Meaning
Distance	15	Starting distance from the hero.
Min / Max Distance	5 / 20	Limits of the mouse wheel zoom.
Zoom Speed / Gamepad Zoom Speed	1.5 / 8	See the V2 Update document.
Horizontal Rotation Speed	20	How fast the camera turns around the hero.
Vertical Rotation Speed	0	0 keeps the camera angle fixed, which gives the top-down roguelike view. Raise it to let the player tilt the camera.
Min / Max Vertical Angle	-50 / 50	Tilt limits when vertical rotation is used.
Collision Layers	Nothing	Layers the camera should not pass through. Select at least Default if you want the camera to stop at walls.
Move Damping	0.5	How smoothly the camera follows the hero.

The **Advanced FPS Camera** on **CameraFPS** has its own rotation speeds and vertical limits (-60 to 40), and it sits on the **CameraFPSPoint** of your hero.

COMBAT AND WEAPONS

Walk to a weapon and press **E** to pick it up. It goes into your inventory and, if that hand is free, it is equipped straight away (see the V2 Update document for the inventory). Press the **Left Mouse Button** to attack. Pressing it again during the swing chains a three-hit combo, and every hit plays its own animation and particle effect. Hold the **Right Mouse Button** to raise your shield: while it is up, enemy hits, arrows and trap damage do not hurt you, and arrows bounce off it.



The hero fighting a soldier with a sword and shield

Weapon	ID	Damage Power	Notes
Sword	1	20	Held in the right hand.
Axe	2	40	Held in the right hand. Deals twice the damage of the sword.
Shield	3	-	Held in the left hand. Used for blocking.

Each weapon prefab has a **Weapon Script** where you set its ID, Name, Weapon Type, Damage Power and the blood particles it spawns. When you hit an enemy, the enemy's name and health bar appear at the top of the screen for a few seconds. Only objects with the `Soldier` tag can be damaged by weapons.

POTIONS

There are three potions in the game. Health and Mana potions are drunk with **1** and **2**, and the Speed potion is used from the inventory. How much each one gives is set on the **Potion Script** of the potion prefab:

Potion	Default value	Effect
Health Potion	Extra_Health 25	Restores 25 health. You can carry four.
Mana Potion	Extra_Mana 50	Restores 50 mana. You can carry four.
Speed Potion	Extra_Speed 4	+4 movement speed for 15 seconds.

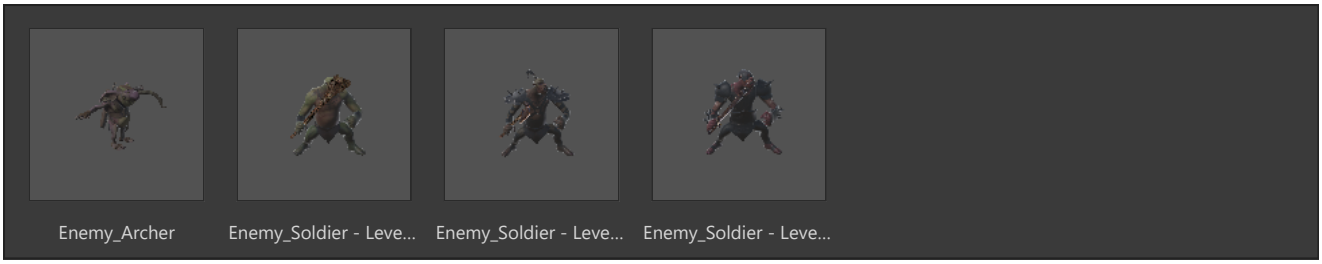


Health and Mana potions in the demo scene

ENEMY AI: SOLDIERS AND ARCHERS

There are two types of enemies with Enemy AI components. You can use them in your game whenever you want. They move with Unity's NavMesh, so the floor of your level needs a baked NavMesh. They have all the basics of an AI system:

- **Idle and Patrol:** the enemy waits for a few seconds. If **Is Patroller** is ticked and **Patrolling_Points** has transforms in it, it walks to a random point. If the list is empty, it stands idle.
- **Realize:** once a second the enemy checks how far your hero is. When you come within its **Range**, it plays an alert animation and sound, then turns to attack you.
- **Chase and Attack:** the Soldier runs to you and swings its weapon when you are closer than 2.5 meters. The Archer stays where it is and shoots arrows at you.
- **Get Damage and Die:** a hit makes the enemy fight back even if it was walking somewhere else. When its health reaches zero it plays the death animation, drops loot and disappears after five seconds.



You can find the enemy prefabs in `Assets\AdvancedRogueLikeandPuzzleSystem\Prefabs\Enemy`. The Soldier uses the **Soldier Controller** component and the Archer uses the **Archer Controller** component. Their default values are:

Prefab	Health	Damage Power	Range	Speed	Attack Period
Enemy_Soldier - Level 1	100	10	15	1.75	2.5
Enemy_Soldier - Level 2	120	15	15	1.75	2.5
Enemy_Soldier - Level 3	150	25	15	1.75	2.5
Enemy_Archer	100	10	10	1.75	2.5

Other useful fields are **Name** (shown on the enemy health bar), the attack, realize and die sound lists, and **Gain**. The **Gain Script** holds a list of loot prefabs: when the enemy dies, it drops nothing, one item or two different items at random. The Archer also needs its **Arrow Prefab** and **Arrow Start Point** (the hand the arrow is held in). Attack Period is randomized a little for every enemy, so a group of enemies never attacks at exactly the same moment.

ENEMY SPAWNER

If you want waves of enemies instead of placing them by hand, use the **EnemySpawner** prefab from the Managers folder. You can try it in the **DemoScene_Spawner** scene.

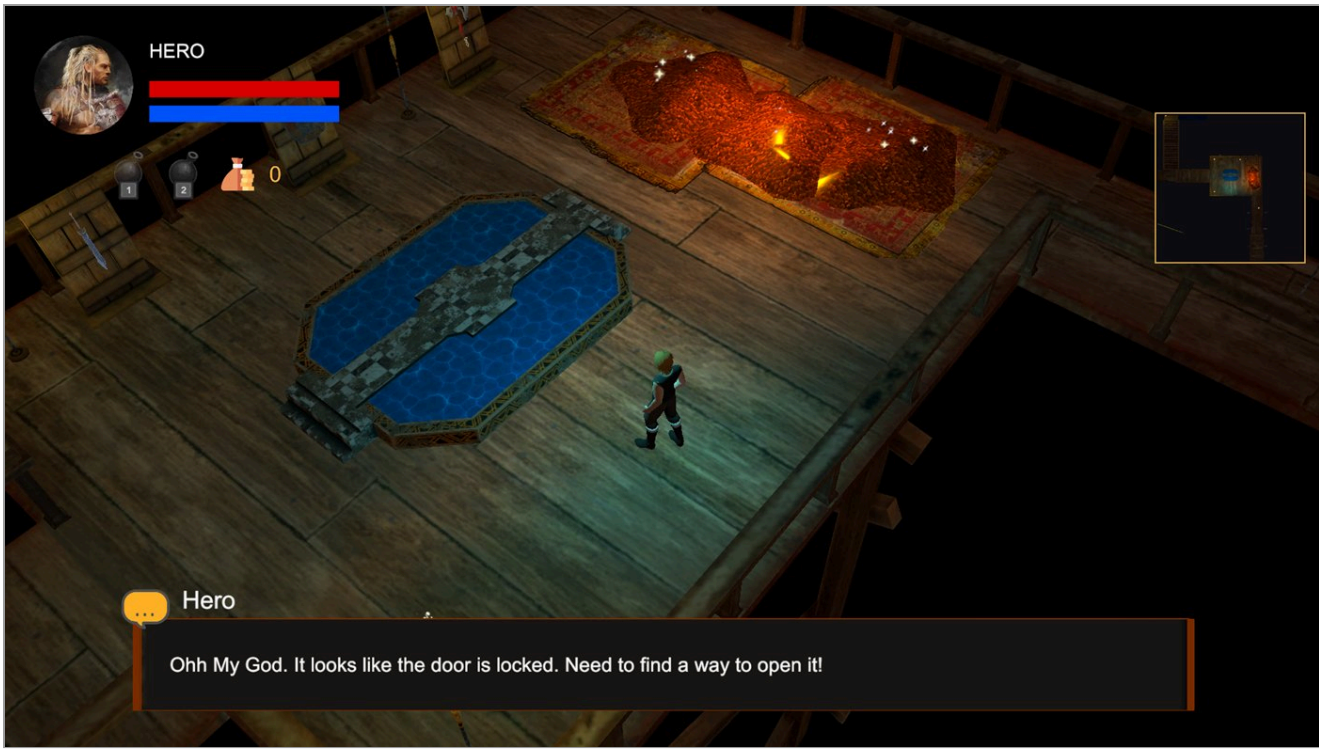


Enemies created by the spawner attacking the hero in DemoScene_Spawner

Field	Default	Meaning
Spawn Interval	1	Seconds between two spawns.
Prefab Types	4 enemies	The spawner picks one of these at random each time.
Target Spawn Count	10	How many enemies are spawned in total.
Max Alive Prefabs	3	The spawner waits while this many enemies are still alive.
Spawn Points	10 points	Where enemies appear. The same point is never used twice in a row.
Can Spawn	On	Turn it off to pause the spawner from your own code.

DIALOGUE SYSTEM

The package has got a dialogue system and it is already integrated. The text is typed letter by letter on the dialogue panel, together with the speaker's name. It closes when the player presses the Interact key, or by itself after 20 seconds.



A dialogue on the game screen

There are two ways to show a dialogue:

Calling It From Your Code

You can call this method from anywhere in your own code with the speech text and the speaker as parameters:

```
SpeechManager.instance.Show_Speach("It looks like the door is locked.", "Hero");
```

- **Speech:** your text which will appear as dialogue.
- **Speaker:** the name of the character who is talking.

Using The Dialogue Trigger Prefab

You can use the **DialogueTrigger** prefab as many times as you want in your scene. Place it where your character should have the dialogue and fill its **Speaker** and **Speech** fields in the inspector. When the hero walks into it, the dialogue is shown once and the trigger removes itself.

TARGET POINTER

You can easily point to an object or location in your game by using the Target Pointer. It is on both cameras (**CameraTPS** and **CameraFPS**), and it draws an arrow on the screen. When the target is off screen, the arrow sits on the edge of the screen and points the way. When the target is on screen, the arrow hovers above it.



The arrow in the top right corner points to a trap that is off screen

You only need to assign your target to the **PointedTarget** variable. For example, if you want to point to a door in your game:

```
TargetPointer.Instance.PointedTarget = exitDoor;
```

If you want to hide the Target Pointer from your game screen, just call this:

```
TargetPointer.Instance.PointedTarget = null;
```

You can change the arrow texture and its **Size** on the Target Pointer component. The size is in screen pixels, so on large screens you may want a bigger value.

DOORS AND KEYS

Doors open and close with the Interact key. Every door has a **Door Script** component:

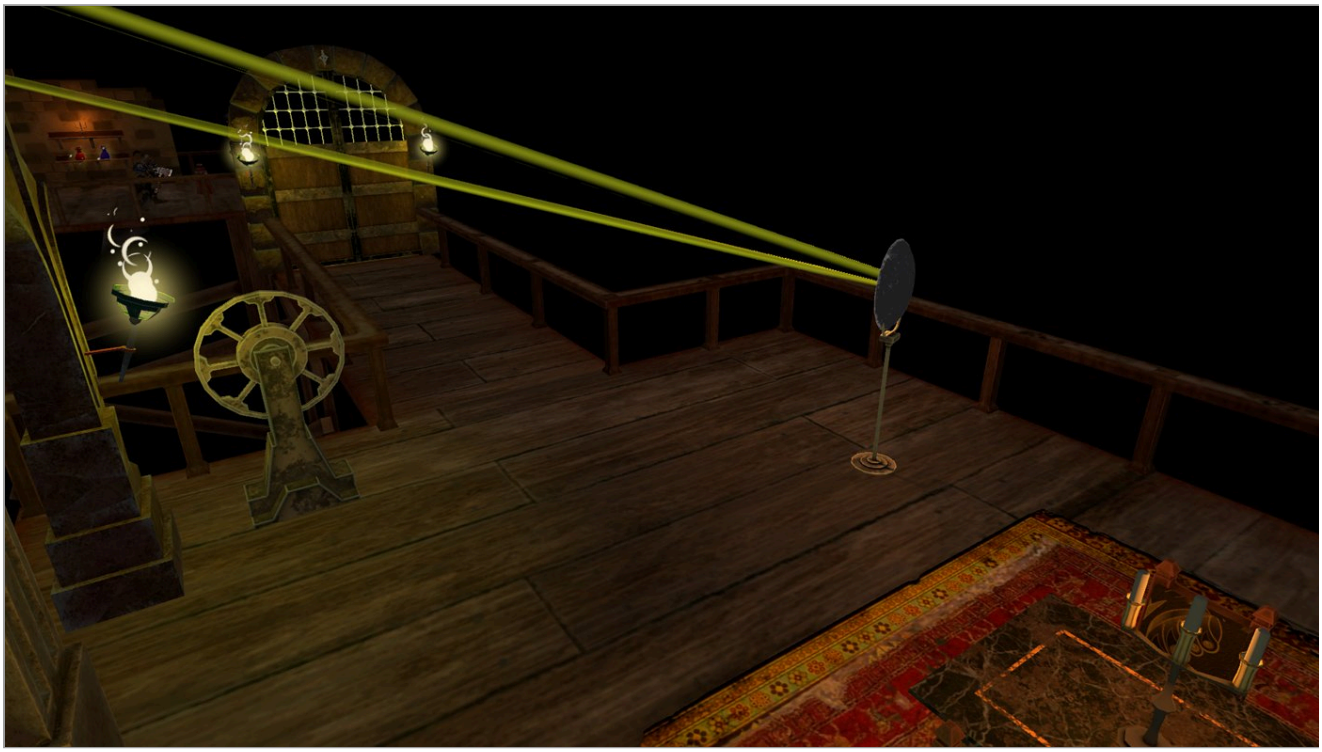
Field	Meaning
Is Locked	A locked door shows "LOCKED!" and shakes until the player has the right key.
Key ID To Open	The Key_ID of the key that opens this door.
Is Closeable	When it is off, the door stays open for good once it is opened.
Door Type	Chooses the animation clips: `Open_Door_<Type>`, `Close_Door_<Type>` and `TryToOpen_<Type>`. The package has Single, Multiple, Multiple2 and Izgara.
Door Material	Wooden or Metal. Chooses the opening sound.
Object Will Be Activated When Door Is Opened	Optional object that is switched on when the door opens, for example the next room's enemies.

The **Key** prefab has a **Key_ID** (1 by default) and a Name. Give a door the same Key ID and the key opens it. In the demo scene the key is hidden inside a chest and opens the Throne Door. Doors can also be opened by a light beam (see Puzzles), by a pressure button, or blown apart by an exploding barrel. The first time each door is opened, the game saves automatically.

PUZZLES

Mirror And Light Puzzle

A beam of light shines into the room. The player walks to the **Gear For Mirror**, presses **E**, and turns the mirror with **A** and **D**. The camera pulls back so the whole room is visible and the hero stands still while the mirror turns. Press **E** again to leave the gear. When the beam hits the mirror, the mirror sends its own beam out, and when a beam reaches a door, the door unlocks and opens.



The gear on the left turns the mirror on the right. The yellow light beam is reflected by the mirror.

- **Gear For Mirror** (**Mirror Gear Script**): assign the **Mirror** it rotates and the **Gear** model that spins with it.

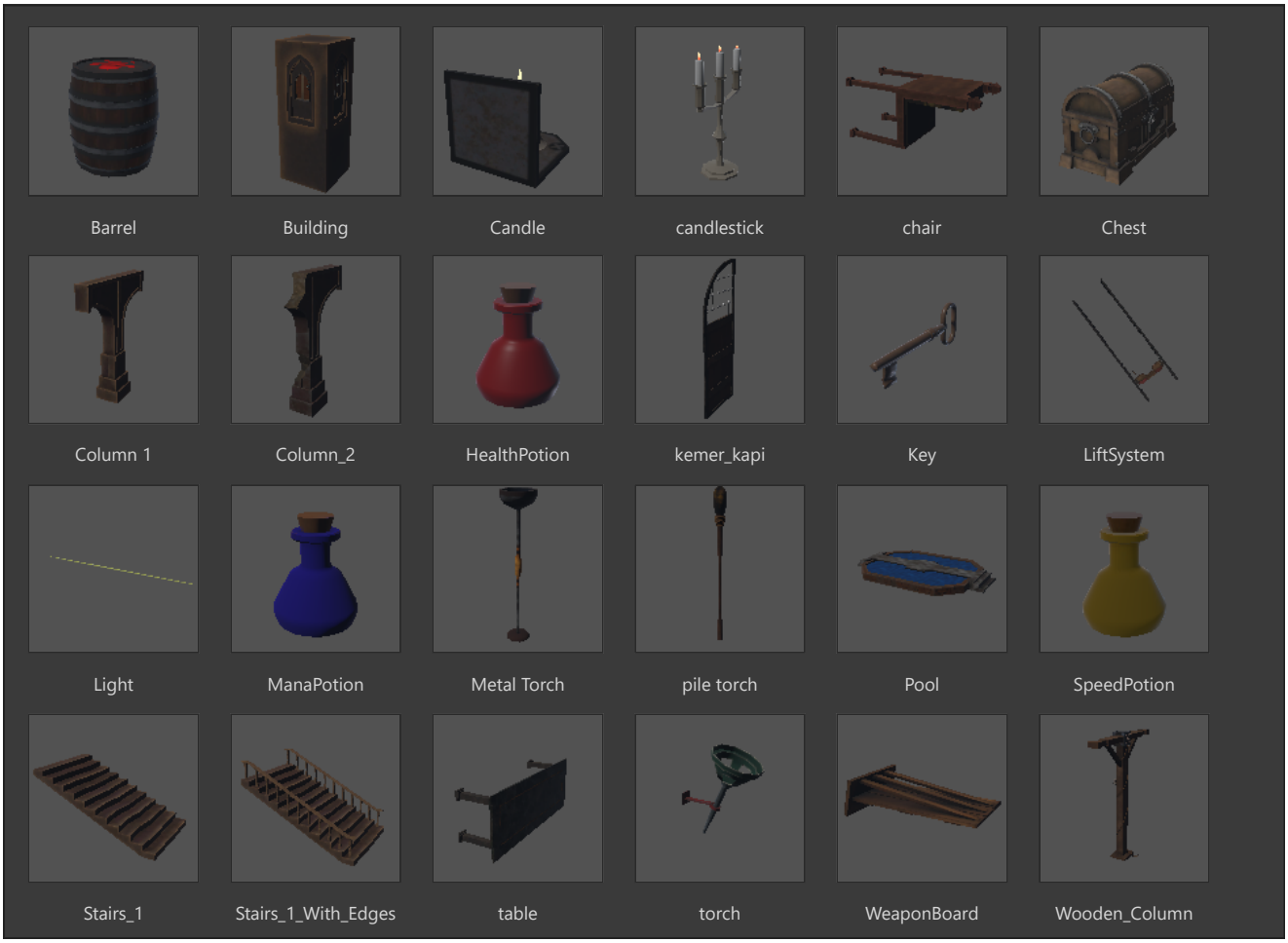
- **Mirror** (**Reflector Script**): **Reflected Light** is the beam this mirror sends out. **Raycast Distance** (20) and **Layers** decide where the beam stops.
- Light beams must have the `Light` tag, and doors react to them through their trigger collider.
- **Puzzle_Mirror** is the complete ready setup from the demo scene.

Pressure Button

The **Button** prefab runs a list of actions once, when the hero steps on it. Fill the **Events To Invoke** list in the inspector like any Unity Event. In the demo scene it calls `UnlockAndOpen` on a stone door. If **Is Interactable** is off, the button does nothing until your code calls `Activate()` on it, which also makes it glow green.

PROPS

The package has got lots of ready to use props to make your game rich. All of them are in the `Assets\AdvancedRogueLikeandPuzzleSystem\Prefabs\Props` directory:



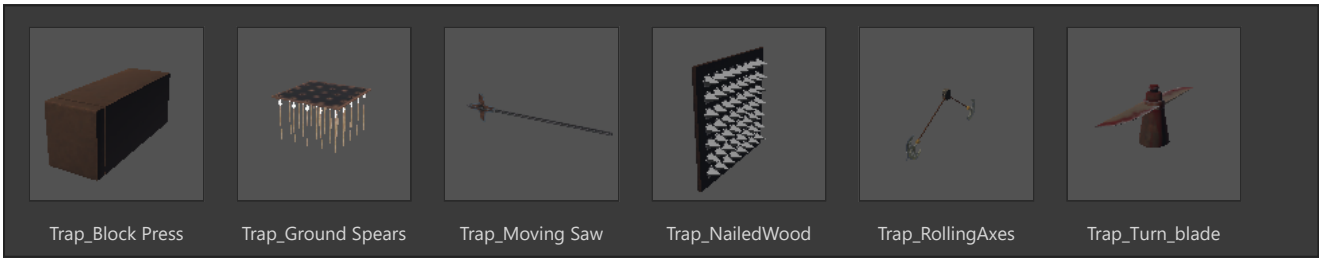
- **Barrel:** when the hero touches it, its fuse starts to burn and it explodes 2.5 to 3.5 seconds later. The explosion deals 20 damage within 5 meters and knocks nearby door pieces (tagged `DoorItem`) loose, so barrels can break doors. You should not wait beside it.
- **Chest:** put objects in it and add them to its **Hidden Objects** list. When you open the chest, these objects appear and you can collect them. Opened chests stay open when you load a saved game.
- **Key:** unlocks doors. See Doors And Keys.
- **LiftSystem:** an elevator between two floors. Walk to the lever and press E, and the platform carries you to the other floor while the gears turn. Just re-position its **UpPoint** and **DownPoint** transforms (the **Top Floor** and **Bottom Floor** fields of the Elevator Script).
- **HealthPotion, ManaPotion, SpeedPotion:** picked up by walking over them. See Potions.
- **WeaponBoard:** a wooden board to hang weapons on, as in the demo scene.
- The rest are decorative props: buildings, columns, stairs, torches, candles, furniture and a pool.



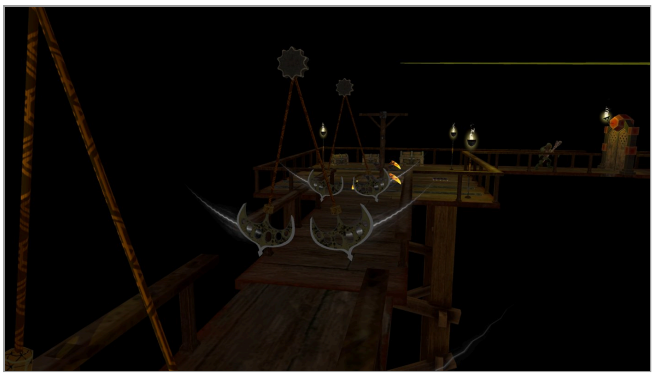
An opened chest with a Mana potion inside

TRAPS

All traps are in the `Assets\AdvancedRogueLikeandPuzzleSystem\Prefabs\Traps` directory. Each one has a **Damage** value on its trap component, and a raised shield blocks their damage too.



Trap	Damage	Behaviour
Trap_Block Press	100	A block goes out and in periodically. If it hits you, you die.
Trap_Ground Spears	50	Half a second after you step on it, spears come out of the floor. If you are still standing on it half a second later, you take damage.
Trap_Moving Saw	100	A rolling saw moves from side to side in a loop. If it touches you, you die.
Trap_NailedWood	100	A board rotates periodically. When its flat side faces up you can stand on it, but you should not be on it when it turns back.
Trap_RollingAxes	100	Two axes swing from side to side. If one of them touches you, you die.
Trap_Turn_blade	100	Two blades rotate around a column. If one of them touches you, you die.



Rolling axes on the bridge



Saws and ground spears in the trap room

MOBILE CONTROLS

Set **Controller Type** to **Mobile** on the Game Manager and the game switches to touch controls. A joystick appears in the bottom left corner, and you look around by swiping on the screen. Attack, shield, sprint, jump, pause and inventory buttons appear on the screen. The Interact button appears only when you stand next to something you can use. Pushing the joystick far to the edge makes the hero run.



The game with Controller Type set to Mobile

The joystick and the touch area come from the **Joystick** and **Touchpad** prefabs in the UI folder and they are already inside the Game Canvas.

TAGS AND LAYERS

The systems recognise objects by their tags, so a new object needs the right tag to work. These tags are already in the project:

Tag	Used for
Player	Your hero. Traps, enemies and triggers look for this tag.
Soldier	Enemies that weapons can hit.
Door	Doors the hero can open.
Weapon	Weapons lying in the world that can be picked up.
Item	Potions, keys, chests, the lift lever and the mirror gear.
Light	Light beams of the mirror puzzle.
DoorItem	Door pieces that barrel explosions blow away.
Collapsable	Debris that fireballs knock down.
Water, WaterForSwimming	Shallow water with a splash, and water the hero can swim in.

The **Minimap** layer is used by the minimap camera, which is described in the V2 Update document.

CONCLUSION

As you can see, there are many features, prefabs, components, an Enemy AI system, weapons, puzzles, props and traps ready to use in this package. You can create your own Roguelike game, Third Person Shooter game or RPG game with this package easily. For the inventory, the save system, the main menu and the minimap, please continue with **Documentation_V2_Update.pdf**.

I hope you like it.

Kind Regards,
Queen Developer